T6 - Matplotlib: bar, line, histogram, scatter, pie, subplots

Covers every chart the exam can ask for and how to build it from a pandas groupby or a plain list. Earns 2024 Task 1b (4 marks), the loss plot of 2024 Task 3d (3 marks), and the "Height Distribution" histogram of Practice Exam Task 3.

1. What the exam asks

2024 final lab exam, Task 1b: Data Visualisation (on `sales_data.csv`)

> **Product Sales Distribution:** Create a bar chart showing the total quantity sold for each product. [2 mark]
>
> **Sales Over Time:** Create a line plot to show the trend of total sales over the months of 2023. The x-axis should represent the months, and the y-axis should represent the total sales. [2 mark]

2024 Task 3d: Loss Visualisation [3 marks]

> Plot the training loss over epochs using matplotlib. Your plot should display the training loss decreasing over time, indicating that the model is learning.

Practice Exam Task 3, Task 4: Visualisation (replaces PyTorch)

> Using matplotlib, create a simple histogram showing the distribution of heights. Title: "Height Distribution". X-axis label: Height (cm). Y-axis label: Frequency.

Where the course taught this: `WEEK3/Class_Lab_5` (`Matplotlib_study_notes.md`, `Lab5 Exercise – solutions.pdf` Tasks 1-19, `Lab5.docx`), and the last two cells of `WEEK3/Class4/Data_Manipulation.ipynb` (bar per product and monthly line chart from a groupby). The marker looks for: **the right chart type, a title, labelled axes, and the right numbers on it.** The 2022 and 2023 Python papers had no charts.

1.1 The instructor's checklist - Matplotlib bullets covered here

> **Matplotlib** (practical-assessment checklist from the course instructor) - creating simple plots such as scatter plots, line plots, bar charts and box plots - adding appropriate titles and axis labels
>
> -> scatter, line, bar: independent templates in **3.11**; box plots: **3.12**; choosing the plot: **3.13**. Titles and both axis labels are in EVERY template.

The instructor's advice: do not memorise one complete solution; practise the *individual operations*. Sections 3.11-3.13 are small independent recipes.

CORRECTION to the "expected numbers" in template K: the real `sales_data.csv` contains 6 exact duplicate rows (04/04/2022 to 10/04/2022 appear twice). The official solution does not remove them in Task 1, and the numbers quoted in template K are with the duplicates left in. If you call `df.drop_duplicates()` first, the quantities become Biscuits 196, Chocolate 120, Orange Juice 232, Sweets 252, Unknown 33, Water 90 (the 2023 monthly chart is unchanged: the duplicates are all in 2022).

2. Mental model (analogy)

- `plt.figure()` **= a fresh sheet of paper** on your desk. Everything you draw goes on the sheet that is on top. Forget to take a new sheet and your second drawing lands on top of the first (the classic overlap bug).
- **Axes = the graph drawn on that sheet** (the box with two number lines). A sheet can hold several graphs: `plt.subplots(1, 2)` = two side by side.
- `plt.plot / bar / hist ...` **= pencil work**; `title / xlabel / ylabel` **= writing the captions**; `legend` **= the key** that says which colour is which.
- `plt.show()` **= hold the sheet up in a window.** Your program waits until you close it. `plt.savefig('x.png')` **= photocopy it to a file** (do it *before* `show`).
- **Which chart?** (course slide "Choosing the right chart"): categories -> **bar**; change over time -> **line**; one list of numbers (spread) -> **histogram**; two numbers per item -> **scatter**; parts of a whole -> **pie**.

3. Copy-paste templates

All assume `import matplotlib.pyplot as plt`, `import pandas as pd`, `import numpy as np`, and a cleaned DataFrame `df` (see T5 template G). The `# 1`... comments show the order: draw, caption, tidy, show.

A. The skeleton every chart follows (Lab5 solutions, Task 1)

```
x = [1, 2, 3, 4]
y = [2, 4, 6, 8]
plt.figure(figsize=(8, 5))        # new sheet, 8in wide x 5in tall
plt.plot(x, y)                    # 1. draw the data
plt.title('Double the number')    # 2. captions: title ...
plt.xlabel('x')                   #    ... x-axis label ...
plt.ylabel('2 x')                 #    ... y-axis label
plt.tight_layout()                # 3. stop labels being cut off
plt.show()                        # 4. display (blocks until closed)
```

B. Bar chart (categories) - totals per product from a groupby

```
qty = df.groupby('Product')['Quantity'].sum()    # Series: product -> total
plt.figure(figsize=(8, 5))
plt.bar(qty.index, qty.values, color='steelblue')   # names, heights
plt.title('Total Quantity Sold per Product')
plt.xlabel('Product')
plt.ylabel('Total Quantity Sold')
plt.xticks(rotation=45)                          # tilt long names
plt.tight_layout()
plt.show()
# horizontal version: plt.barh(qty.index, qty.values) + swap the labels
```

C. Line chart (change over time) - monthly totals for ONE year

```
sales_2023 = df[df['Date'].dt.year == 2023]        # 1. keep 2023 rows only
monthly = sales_2023.groupby(sales_2023['Date'].dt.month)['Total'].sum()
names = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
         'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
plt.figure(figsize=(8, 5))
plt.plot(monthly.index, monthly.values, marker='o', color='darkgreen')
plt.xticks(monthly.index, [names[m - 1] for m in monthly.index])  # Jan, Feb..
plt.title('Total Sales per Month (2023)')
plt.xlabel('Month')
plt.ylabel('Total Sales')
plt.grid(True, linestyle='--', alpha=0.5)          # light grid helps reading
plt.tight_layout()
plt.show()
```

D. Histogram (spread of one list of numbers) and **bins**

```
ages = [23, 25, 31, 35, 36, 38, 41, 44, 45, 52, 58, 61]
plt.figure()
plt.hist(ages, bins=5, edgecolor='black')   # 5 equal-width buckets
plt.title('Age Distribution')
plt.xlabel('Age')
plt.ylabel('Frequency')                     # y = HOW MANY fall in each bucket
plt.show()
# bins=5 coarse/smooth   bins=20 detailed/noisy   default = 10
# bins=[20, 30, 40, 50, 60, 70] = your own bucket edges
```

E. Scatter (two numbers per item) and pie (parts of a whole)

```
plt.figure()
plt.scatter(df['Quantity'], df['Total'], alpha=0.5)   # one dot per row
plt.title('Quantity vs Total')
plt.xlabel('Quantity')
plt.ylabel('Total')
plt.show()

share = df.groupby('Product')['Total'].sum()
```

```
plt.figure(figsize=(6, 6))
plt.pie(share, labels=share.index, autopct='%1.1f%%', startangle=90)
plt.title('Share of Total Sales')                    # %1.1f%% = "12.3%"
plt.show()
```

F. Two charts in one script - give each its own `plt.figure()`

```
plt.figure()                         # sheet 1
plt.bar(['A', 'B', 'C'], [5, 7, 3])
plt.title('Chart one')

plt.figure()                         # sheet 2: a NEW window, no overlap
plt.plot([1, 2, 3], [3, 1, 2], marker='o')
plt.title('Chart two')

plt.show()                           # ONE show() at the end shows both windows
```

G. Subplots: `fig, ax = plt.subplots()` (the style of Lab5.docx; two charts on ONE sheet)

```
fig, ax = plt.subplots(1, 2, figsize=(10, 4))   # 1 row, 2 columns
ax[0].bar(qty.index, qty.values)                # left chart
ax[0].set_title('Quantity per product')         # NOTE: set_title, not title
ax[0].set_xlabel('Product')
ax[0].set_ylabel('Quantity')
ax[0].tick_params(axis='x', rotation=45)        # tilt the left labels
ax[1].plot(monthly.index, monthly.values, marker='o')   # right chart
ax[1].set_title('Sales per month (2023)')
ax[1].set_xticks(monthly.index)                 # whole months only
ax[1].set_xlabel('Month')
ax[1].set_ylabel('Total Sales')
fig.tight_layout()
plt.show()
# one chart only: fig, ax = plt.subplots() then ax.plot(..) ax.set_title(..)
# 2 rows x 1 col:  plt.subplots(2, 1)   2x2 grid: ax[row, col]
```

H. The pandas shortcut `df.plot(kind=...)` (what the course solution used: pandas draws it with matplotlib for you)

```
qty.plot(kind='bar', title='Total Quantity per Product',
         xlabel='Product', ylabel='Total Quantity', color='orange')
plt.tight_layout()
plt.show()                                      # still need show()

df['Quantity'].plot(kind='hist', bins=7, edgecolor='black',
                    title='Quantity distribution')
plt.xlabel('Quantity')
plt.show()
# kinds: 'bar' 'barh' 'line' 'hist' 'pie' 'scatter' (scatter needs x=, y=)
```

I. Plot a groupby result - month charts straight from a Series (the course idiom: `Data_Manipulation.ipynb`, "Month" via `to_period`)

```
d23 = df[df['Date'].dt.year == 2023].copy()      # filter the year FIRST
d23['Month'] = d23['Date'].dt.to_period('M')     # 2023-01, 2023-02 ...
by_month = d23.groupby('Month')['Total'].sum()   # index = month periods
by_month.plot(kind='line', marker='o', title='Total Sales 2023')
plt.xlabel('Month')
plt.ylabel('Total Sales')
plt.tight_layout()
plt.show()
```

J. Several lines + legend + training-loss list (2024 Task 3d; Lab5 Task 3)

```
losses = [0.91, 0.74, 0.60, 0.51, 0.44, 0.39, 0.35, 0.32]   # one per epoch
epochs = range(1, len(losses) + 1)               # 1, 2, ... 8
plt.figure()
plt.plot(epochs, losses, marker='o', color='red', label='Training loss')
plt.title('Training Loss over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
```

```
plt.legend()                                    # shows the label= text
plt.grid(True)
plt.show()
# in PyTorch: losses.append(loss.item()) inside the epoch loop; then plot.
# several series: call plt.plot(..., label='A') then plt.plot(..., label='B'),
# then ONE plt.legend().  plt.legend(loc='upper right') chooses the corner.
```

K. EXAM Task 1b - paste-ready (cleaning lines included so it is self-contained; tested on `sales_data.csv`, 2023 = Jan to 10 Apr)

```
import pandas as pd
import matplotlib.pyplot as plt

# ---- Task 1a (from T5): load and clean ----
df = pd.read_csv('sales_data.csv')
rec = df['Price'].isna() & df['Total'].notna() & (df['Quantity'] > 0)
df.loc[rec, 'Price'] = df['Total'] / df['Quantity']       # recover Price
df = df.fillna({'Product': 'Unknown', 'Quantity': 0,
                'Price': 0.0, 'Total': 0.0})
df['Date'] = pd.to_datetime(df['Date'], dayfirst=True)    # dd/mm/yyyy
df['Total'] = df['Quantity'] * df['Price']

# ---- Task 1b (i): bar chart, total quantity sold per product [2] ----
qty_per_product = df.groupby('Product')['Quantity'].sum()

plt.figure(figsize=(8, 5))
plt.bar(qty_per_product.index, qty_per_product.values, color='steelblue')
plt.title('Total Quantity Sold per Product')
plt.xlabel('Product')
plt.ylabel('Total Quantity Sold')
plt.xticks(rotation=45)
plt.tight_layout()

# ---- Task 1b (ii): line chart, total sales per month of 2023 [2] ----
sales_2023 = df[df['Date'].dt.year == 2023]               # only 2023
monthly = sales_2023.groupby(sales_2023['Date'].dt.month)['Total'].sum()
names = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun',
         'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']

plt.figure(figsize=(8, 5))                                # NEW figure
plt.plot(monthly.index, monthly.values, marker='o', color='darkgreen')
plt.xticks(monthly.index, [names[m - 1] for m in monthly.index])
plt.title('Total Sales per Month (2023)')
plt.xlabel('Month')
plt.ylabel('Total Sales')
plt.grid(True, linestyle='--', alpha=0.5)
plt.tight_layout()

plt.show()                        # shows both figures (two windows)
```

Expected numbers (check your chart against these): quantity per product - Biscuits 203, Chocolate 120, Orange Juice 248, Sweets 261, Unknown 33, Water 96. Monthly 2023 totals - Jan 2735, Feb 2535, Mar 2785, Apr 895. **NOTE:** April is low only because the file ends on 10 April 2023; the "Unknown" bar is the six sales whose Product was blank.

L. Practice Exam Task 3 - the "Height Distribution" histogram (needs `heights.csv`; the T3 document creates it)

```
import pandas as pd
import matplotlib.pyplot as plt

df = pd.read_csv('heights.csv')                    # columns: Name, Height
plt.figure()
plt.hist(df['Height'], bins=5, edgecolor='black')  # bars touch = a histogram
plt.title('Height Distribution')                   # EXACT wording asked
plt.xlabel('Height (cm)')
plt.ylabel('Frequency')
plt.show()                               # or plt.savefig('hist.png')
```

M. `show` or `savefig`?

```
plt.savefig('chart.png')              # to a file (type from extension)
plt.savefig('chart.png', dpi=300, bbox_inches='tight')   # sharper, trimmed
plt.show()                                    # AFTER savefig, never before
```

- The exam says "plot", so `plt.show()` is what is marked. Add a `savefig` line only if the question says "save".
- With several figures, `plt.savefig` saves the *current* (latest) one; use `fig.savefig(...)` on a named figure (`fig = plt.figure()`).

3.11 Scatter, line and bar: three independent minimal templates

Each one stands alone (own data, own captions). Swap in your own lists or DataFrame columns.

Scatter - is there a relationship between two numbers?

```
hours = [1, 2, 3, 4, 5, 6, 7, 8]           # x: hours studied
marks = [35, 42, 50, 55, 62, 70, 74, 85]   # y: mark achieved (same length!)
plt.figure()
plt.scatter(hours, marks, color='purple', s=60, alpha=0.7, marker='o')
plt.title('Study Time vs Exam Mark')
plt.xlabel('Hours studied')
plt.ylabel('Exam mark')
plt.show()
```

Part	What it does
`scatter(x, y)`	one dot per (x, y) pair; no line joins them
`color=`	dot colour (`'red'`, `'purple'`, `'#1f77b4'`)
`s=60`	dot SIZE (area); bigger number = bigger dot
`alpha=0.7`	transparency, 0 (invisible) to 1 (solid); helps when dots overlap
`marker='o'`	dot shape (`'s'` square, `'^'` triangle, `'x'` cross)
`label=`	name for `plt.legend()` when you draw several scatters

Line - how does a value change over time/order?

```
months = [1, 2, 3, 4, 5, 6]
sales = [210, 180, 225, 220, 205, 260]
plt.figure()
plt.plot(months, sales, color='darkgreen', marker='o',
         linestyle='--', linewidth=2, label='Sales')
plt.title('Monthly Sales')
plt.xlabel('Month')
plt.ylabel('Units sold')
plt.legend()
plt.grid(True)
plt.show()
```

Part	What it does
`plot(x, y)`	joins the points IN THE ORDER GIVEN (sort by time first)
`marker='o'`	a dot at every data point
`linestyle='--'`	`'-'` solid, `'--'` dashed, `':'` dotted
`linewidth=2`	line thickness
`label='Sales'` + `plt.legend()`	key box; no `legend()` call = no key
`plt.grid(True)`	light grid to read values

Bar - compare categories

```
products = ['Water', 'Sweets', 'Biscuits']
totals = [720, 3915, 4060]
plt.figure()
plt.bar(products, totals, color='steelblue', width=0.6, edgecolor='black')
plt.title('Total Sales per Product')
plt.xlabel('Product')
```

```
plt.ylabel('Total sales')
plt.show()
# horizontal: plt.barh(products, totals) — then xlabel is the number axis
```

Part	What it does
`bar(names, heights)`	one bar per category; names on x, heights on y
`color=`	one colour, or a list with one colour per bar
`width=0.6`	bar width (default 0.8; smaller = bigger gaps)
`edgecolor='black'`	outline, nice on printed paper
`plt.barh(names, widths)`	horizontal bars: good for long names
`yerr=`	error bars (one value per bar)

3.12 Box plots

Analogy: a box plot is a **shrink-wrapped summary of a pile of numbers**. Line up all the values from smallest to largest, cut the row into four equal groups, and draw the middle two groups as a box. The line across the box is the middle person in the row (the median).

How to read one:

Part	Meaning
line inside the box	**median** (50% of values lie below it)
bottom / top of box	**Q1** (25th percentile) and **Q3** (75th percentile)
box height	**IQR** = Q3 - Q1, the spread of the middle 50%
whiskers	reach to the furthest value within 1.5 x IQR of the box
separate dots beyond whiskers	**outliers** (unusually high/low values)
tall box / long whiskers	values very spread out; short box = consistent

Rule of thumb: compare boxes by **median** (height of the line), **spread** (box height) and **outliers** (dots).

```
class_a = [55, 60, 62, 65, 67, 70, 72, 75, 78, 95]
class_b = [40, 48, 55, 58, 60, 62, 65, 70, 80, 85]
class_c = [66, 68, 69, 70, 71, 72, 72, 73, 74, 75]

plt.figure(figsize=(6, 4))
plt.boxplot([class_a, class_b, class_c])      # a LIST of lists: one box each
plt.xticks([1, 2, 3], ['Class A', 'Class B', 'Class C'])   # name the boxes
plt.title('Exam Marks by Class')
plt.xlabel('Class')
plt.ylabel('Mark')
plt.tight_layout()
plt.show()
# reading it: Class A has the highest median and ONE outlier (95);
# Class B is the most spread out; Class C is very consistent (short box).
```

Naming the boxes - version warning (tested on Matplotlib 3.11):

```
plt.boxplot(data, tick_labels=['A','B'])   # Matplotlib 3.9 and newer: works
plt.boxplot(data, labels=['A','B'])        # OLD name. In 3.11 this gives
   TypeError: boxplot() got an unexpected keyword argument 'labels'
plt.boxplot(data, vert=False)              # old horizontal switch
   warns it is deprecated; use orientation='horizontal'
```

The safe way that works in EVERY version: leave the keyword out and call `plt.xticks([1, 2, 3], [names])` after `plt.boxplot(..)`, as above. Boxes are numbered 1, 2, 3, ... (not 0).

Box plot straight from a DataFrame (the long format of Lab5.docx Task 12: one column of groups, one of values)

```
marks = pd.DataFrame({'Class': ['A'] * 10 + ['B'] * 10 + ['C'] * 10,
                      'Mark': class_a + class_b + class_c})
marks.boxplot(column='Mark', by='Class', grid=False, figsize=(6, 4))
plt.suptitle('')                # remove pandas' automatic extra title
```

```
plt.title('Exam Marks by Class')
plt.xlabel('Class')
plt.ylabel('Mark')
plt.show()

# one box per COLUMN of a DataFrame:
wide = pd.DataFrame({'Class A': class_a, 'Class B': class_b,
                     'Class C': class_c})
wide.plot(kind='box', title='Exam Marks by Class')
plt.ylabel('Mark')
plt.show()

df = pd.read_csv('sales_data.csv')            # a single column: one box
df['Total'].plot(kind='box', title='Total per sale')
plt.ylabel('Total')
plt.show()
```

- `column='Mark'` = the numbers to summarise; `by='Class'` = make one box per distinct value of this column.
- Boxes for the sales file look flat when split by Product, because every product has one fixed price: that is the data, not a mistake.
- Quartiles of the sales `Total` column: 50 / 100 / 135 (Q1 / median / Q3).

3.13 Which plot for which question?

Question	Plot	Call
Are two numbers related? (hours vs mark)	**scatter**	`plt.scatter(x, y)`
How does it change over time / order? (monthly sales)	**line**	`plt.plot(x, y)`
How do categories compare? (total per product)	**bar**	`plt.bar(names, heights)`
How are the values spread? (heights of 30 students)	**histogram**	`plt.hist(values, bins=..)`
How do SEVERAL groups differ in spread, median and outliers?	**box plot**	`plt.boxplot([g1, g2, g3])`
What share of a whole does each part take?	pie	`plt.pie(parts, labels=..)`

Clue words: "relationship / correlation / against" = scatter; "trend / over time / each month" = line; "compare / total per / for each" = bar; "distribution / frequency" = histogram; "spread / outliers / median per group" = box.

4. Line-by-line walkthrough (Task 1b)

```
qty_per_product = df.groupby('Product')['Quantity'].sum()
# groupby = one pile per product; ['Quantity'].sum() adds each pile.
# Result is a Series: index = product names, values = totals.

plt.figure(figsize=(8, 5))     # take a new sheet; bigger = labels fit
plt.bar(qty_per_product.index, # x: the five/six product names
        qty_per_product.values,# height of each bar = total quantity
        color='steelblue')     # optional colour
plt.title('Total Quantity ...')# caption on top: WHAT the chart shows
plt.xlabel('Product')          # what the bottom axis means
plt.ylabel('Total Quantity..') # what the side axis means (include the unit)
plt.xticks(rotation=45)        # tilt names so "Orange Juice" fits
plt.tight_layout()             # trim margins so nothing is clipped

sales_2023 = df[df['Date'].dt.year == 2023]   # boolean filter: 2023 only
# group by the MONTH NUMBER (1,2,3,4) taken from each row's date:
monthly = sales_2023.groupby(sales_2023['Date'].dt.month)['Total'].sum()

plt.figure(figsize=(8, 5))     # SECOND sheet: otherwise it draws over the bar
plt.plot(monthly.index,        # x: 1,2,3,4 (months)
         monthly.values,       # y: total sales that month
         marker='o')           # a dot on every month so single points show
plt.xticks(monthly.index, [names[m - 1] for m in monthly.index])
# ^ replace tick numbers 1..4 with 'Jan'..'Apr'  (names[0] is 'Jan')
plt.show()                     # open both windows; program pauses here
```

Why filter the year first? The file holds 2022, 2023 AND 2024. If you group by month number only (or by `to_period('M')` on everything) the question "months of 2023" is not answered. The course solution (`ProgSD-Python-Solution.py`) groups every row by `to_period('M')`, so its line chart covers March 2022 to April 2024 - a reasonable chart, but not the one the question asked for.

Why `.dt.month` and not `.dt.to_period('M')` here? A Period index cannot be handed to `plt.plot` (see traps). Month numbers are plain numbers, so `plt.plot` accepts them. If you prefer the course idiom use template I, where pandas draws the chart itself.

5. Traps & debugging

Layout and display

Mistake	What you see	Fix
Second chart without `plt.figure()`	both charts drawn on the same axes (bars with a line on top)	start each chart with `plt.figure()`
`plt.show()` after every chart	works, but the second chart only appears after you close the first	one `plt.show()` at the end (or accept it)
`plt.savefig` *after* `plt.show()`	empty/blank image file	`savefig` first
x labels overlap / cut-off axis title	squashed or hidden text	`plt.xticks(rotation=45)` and `plt.tight_layout()`
missing title / axis labels	lost style marks	the 3 lines `title`, `xlabel`, `ylabel` every time
`ax[0]` on a single `plt.subplots()`	`TypeError: 'Axes' object is not subscriptable`	one plot: use `ax.plot`; indexes only when 2+ plots
`ax.title('..')` / `ax.xlabel('..')`	`TypeError: 'Text' object is not callable` / `AttributeError: ... no attribute 'xlabel'`	Axes methods are `set_title`, `set_xlabel`, `set_ylabel`

Data traps

Mistake	Result	Fix
`plt.bar(df['Product'], df['Quantity'])` with no groupby	151 bars drawn on top of each other, only the tallest shows	`groupby('Product')['Quantity'].sum()` first
`plt.plot(period_index, values)`	`TypeError: float() argument must be a string or a real number, not 'Period'`	`plt.plot(series.index.astype(str), series.values)` or `series.plot()`
months grouped by name `dt.strftime('%b')`	x order becomes Apr, Feb, Jan, Mar (alphabetical)	group by `dt.month` (a number), then relabel with `xticks`
`plt.plot(x, y)` with different lengths	`ValueError: x and y must have same first dimension`	lengths must match (`len(x) == len(y)`)
`plt.bar(['a','b'], [1,2,3])`	`ValueError: shape mismatch`	one height per name
chart looks wrong because blanks were never filled	wrong totals (e.g. Feb 2400 instead of 2535)	do Task 1a cleaning first; recompute Total
`pd.to_datetime(df['Date'])` without `dayfirst=True`	`ValueError` in pandas 3 (or wrong months)	`dayfirst=True` (T5)
histogram of a text column	`TypeError` / garbage	`hist` needs numbers: `df['Height']`
`plt.pie` with a Series containing NaN	`ValueError: Wedge sizes must be finite numbers`	`dropna()` first

Version / environment notes

- Windows in the exam: if no window appears, make sure the script *ends* with `plt.show()`; in Spyder/Jupyter the chart may appear in a plots pane.
- `df.plot(...)` returns an Axes: `ax = df.plot(...)` then `ax.set_ylabel(..)` works the same as `plt.ylabel(..)`.

- The `xlabel=` / `ylabel=` keyword arguments of `df.plot` exist in current pandas; if an old pandas rejects them, call `plt.xlabel(..)` afterwards.
- `plt.hist` ignores NaN silently, `plt.plot` leaves a gap - a chart that looks fine can still sit on blanks; clean first.

5.1 Traps for sections 3.11-3.13

Mistake	What you see	Fix
`plt.boxplot(data, labels=[..])`	Matplotlib 3.11: `TypeError: boxplot() got an unexpected keyword argument 'labels'`	`tick_labels=` (3.9+) or `plt.xticks([1,2,3], [names])`
`plt.boxplot(df['A'], df['B'])`	second argument is read as `notch`: `ValueError: truth value of a Series is ambiguous`	ONE list of datasets: `plt.boxplot([df['A'], df['B']])`
`plt.boxplot(df)` on a table with text columns	`TypeError`/garbage	pick numeric columns: `df[['A','B']]`
box plot of data containing NaN	that box silently disappears (median is NaN)	`df['A'].dropna()` first
`df.boxplot(column='Mark', by='Class')` extra grey title "Boxplot grouped by Class"	untidy	`plt.suptitle('')` then your own `plt.title`
xticks for boxes start at 0	labels shifted by one	boxes sit at 1, 2, 3
`vert=False`	deprecated warning on Matplotlib 3.11	`orientation='horizontal'`
scatter with different-length lists	`ValueError: x and y must be the same size`	check `len(x) == len(y)`
scatter with the points joined by a line wanted	`scatter` draws no lines	use `plt.plot(x, y, marker='o')`
line chart looks zig-zag	data not in time order	sort by x before `plt.plot`
`plt.legend()` shows nothing; warning "No artists with labels found"	forgot `label=` in the plot call	`plt.plot(..., label='A')` then `plt.legend()`
bar `color=['red','blue']` with 5 bars	colours repeat	one colour per bar
using a scatter to show "total per product"	meaningless dots	categories -> bar (table 3.13)

6. Anticipated exam questions

Q1 (2) Create a bar chart showing the total quantity sold for each product. Template B (groupby, `plt.bar`, title, both labels).

Q2 (2) Line plot of total sales per month of 2023; x = months, y = total sales. Template C (filter year, groupby `dt.month`, `plt.plot`).

Q3 (2) Show both charts in the same program without them overlapping. Call `plt.figure()` before each chart and `plt.show()` once at the end (template F), or use two separate `fig, ax = plt.subplots()` calls.

Q4 (3) Plot the training loss recorded for each epoch; the x-axis is the epoch number.

```
plt.figure()
plt.plot(range(1, len(losses) + 1), losses, marker='o')
plt.title('Training Loss over Epochs')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.show()
```

Q5 (3) (Practice Task 3) Histogram of heights with the given title and axis labels. Template L; mention `bins=` changes the bucket width.

Q6 (2) Which chart suits "the share of total sales per product" and why? A pie (parts of a whole) - but a bar chart is easier to read exact values from. Pie code: template E. Say both.

Q7 (3) Draw the bar chart and the monthly line chart side by side in one window. Template G (`plt.subplots(1, 2)`; `ax[0]`, `ax[1]`; `set_title`).

Q8 (2) Save the chart as `sales.png` as well as showing it. `plt.savefig('sales.png')` immediately before `plt.show()`.

Q9 (3) Show total sales per month for 2023 AND 2024 as two lines on one chart with a legend.

```
for year in [2023, 2024]:
    d = df[df['Date'].dt.year == year]
    m = d.groupby(d['Date'].dt.month)['Total'].sum()
    plt.plot(m.index, m.values, marker='o', label=str(year))
plt.title('Monthly Sales by Year')
plt.xlabel('Month')
plt.ylabel('Total Sales')
plt.legend()
plt.show()
```

Q10 (2) Why did you use `marker='o'`? With few points (only 4 months) the dots show where the data really is; the line only joins them.

6.1 Extra questions on the instructor's checklist

Q11 (2) Draw a scatter plot of hours studied against exam mark with a title and both axis labels. Template 3.11 (scatter). `plt.scatter(hours, marks)`; title; `xlabel('Hours studied')`; `ylabel('Exam mark')`.

Q12 (3) Draw box plots comparing the marks of three classes. Label the boxes and axes. Template 3.12:

```
plt.figure()
plt.boxplot([class_a, class_b, class_c])
plt.xticks([1, 2, 3], ['Class A', 'Class B', 'Class C'])
plt.title('Exam Marks by Class')
plt.xlabel('Class')
plt.ylabel('Mark')
plt.show()
```

Q13 (3) What does the box plot show and which class has the most consistent marks? Median line, quartile box, whiskers, outlier dots. Class C: the shortest box (smallest IQR) = most consistent; Class A has an outlier (95).

Q14 (2) A box plot of heights shows one dot far above the whisker. What is it? An outlier: a value beyond Q3 + 1.5 x IQR. (Check the data: a typo or a genuinely tall student?)

Q15 (2) Which chart type would you use for (a) monthly trend, (b) relationship between two numbers, (c) spread per group (d) totals per category? Say why. line; scatter; box plot; bar (table 3.13).

Q16 (3) Draw a line chart of `months` and `sales` with a dashed line, circle markers, a legend and a grid. Template 3.11 (line).

Q17 (3) Make a box plot of `Mark` for each `Class` straight from the DataFrame `marks`. `marks.boxplot(column='Mark', by='Class')`, then `plt.suptitle('')`, title, labels, `plt.show()`.

7. Quick-reference box (hand-copy this)

```
IMPORT   import matplotlib.pyplot as plt
NEW PAGE plt.figure(figsize=(8,5))    <- one per chart, or charts overlap!
BAR      s = df.groupby('P')['Q'].sum()      plt.bar(s.index, s.values)
LINE     plt.plot(x, y, marker='o')          label='A' + plt.legend()
HIST     plt.hist(values, bins=5, edgecolor='black')
SCATTER  plt.scatter(x, y)
PIE      plt.pie(s, labels=s.index, autopct='%1.1f%%')
CAPTIONS plt.title('..')  plt.xlabel('..')  plt.ylabel('..')
TIDY     plt.xticks(rotation=45)    plt.tight_layout()    plt.grid(True)
OUTPUT   plt.savefig('a.png')  THEN  plt.show()      (savefig BEFORE show)
SUBPLOT  fig, ax = plt.subplots(1, 2, figsize=(10,4))
         ax[0].bar(..)  ax[0].set_title('..')  ax[0].set_xlabel('..')
         fig.tight_layout()
PANDAS   s.plot(kind='bar'|'line'|'hist'|'pie', title='..')  then plt.show()
MONTHS   d = df[df['Date'].dt.year==2023]
         m = d.groupby(d['Date'].dt.month)['Total'].sum()   (x = 1..12)
LOSS     plt.plot(range(1, len(losses)+1), losses)  + title/Epoch/Loss labels
HEIGHTS  title 'Height Distribution'  x 'Height (cm)'  y 'Frequency'
TRAPS    plot(PeriodIndex) fails -> .astype(str) | groupby BEFORE bar
--- instructor checklist additions ---
SCATTER  plt.scatter(x, y, color=, s=size, alpha=, marker='o')  2 numbers/item
LINE     plt.plot(x, y, marker='o', linestyle='--', linewidth=2, label='A')
BAR      plt.bar(names, heights, color=, width=0.6, edgecolor='black')
BOX      plt.boxplot([g1, g2, g3])   plt.xticks([1,2,3], ['A','B','C'])
         NOT labels=  (gone in Matplotlib 3.11); tick_labels= = 3.9+ only
         df.boxplot(column='V', by='G'); plt.suptitle('')
         df.plot(kind='box')    boxes sit at 1,2,3...
READ BOX line=median, box=Q1..Q3 (IQR), whiskers=1.5*IQR, dots=outliers
WHICH    trend->line  relation->scatter  compare->bar  spread->hist
         spread/outliers per group->box   parts of whole->pie
EVERY plot: plt.title('..')  plt.xlabel('..')  plt.ylabel('..')
```